# Notation Crosswalk
# 2023 Paper, 2025 Paper, and Current Codebase

Internal repository note

March 21, 2026

## 1 Purpose

This note maps the mathematical notation in the 2023 paper, the 2025 paper, and the current Python implementation. It is organized by *concept*, not by symbol, so that semantic mismatches are visible instead of hidden behind renaming.

The goal is practical: when extending the repository, we want to know which parts are simple notation changes and which parts are true model changes.

## 2 Sources

- 2023 paper transcription: `docs/papers/2023`, file `AdaptiveFractionation_2023Paper.tex`
- 2025 paper transcription: `docs/papers/2025`, file `Overlap_Adaptive_Fractionation_2025Paper.tex`
- Core solver: package `adaptive_fractionation_overlap`, module `core_adaptfx.py`
- Helper functions: package `adaptive_fractionation_overlap`, module `helper_functions.py`
- Constants: package `adaptive_fractionation_overlap`, module `constants.py`

## 3 Global Crosswalk

| **Concept** | **2023 paper** | **2025 paper** | **Current codebase** |
|---|---|---|---|
| Geometry descriptor | Sparing factor $\delta_t = d_t^N / d_t$ | Overlap volume $o_t$ in cc | `volumes`, `actual_volume`, `volume_space` |
| Per-fraction decision variable | Tumor dose $d_t$ | PTV dose $d_t$ | `physical_dose`, candidate actions `delivered_doses` |
| Accumulated treatment state | OAR BED $B_N$; appendix also tumor BED $B_T$ | Cumulative delivered physical dose $D_t = \sum_{\tau<t} d_\tau$ | `accumulated_dose`; discretized as `dose_space` |

| Concept | 2023 paper | 2025 paper | Current codebase |
|---|---|---|---|
| Primary objective | Maximize expected tumor BED10 subject to OAR BED3 limit | Minimize expected cumulative overlap-based underdosage cost subject to exact total PTV dose | Maximize negative penalty, which is equivalent to minimizing a penalized overlap cost under an exact total physical-dose target |
| Immediate stage term | Reward $r_t = d_t + d_t^2/(\alpha/\beta)_T$ | Cost $c_t = o_t(d_t - d_{\min})$ | Penalty $\hat{c}_t = o_t(d_t - d_{\min}) + \frac{1}{2}\lvert b + m o_t\rvert (d_t - d_{\min})^2$ via `penalty_calc_*` |
| Terminal condition | $-\infty$ if final OAR BED exceeds $B_N^{\max}$ | 0 if total dose reaches $D_{\text{pres}}$, $\infty$ otherwise | Large negative sentinel value if underdose or overdose occurs; exact goal enforced by hard penalty |
| Probability model | $\delta_t \sim \mathcal{N}(\mu, \sigma^2)$, truncated at 0 in prose | $o_t \sim \mathcal{N}(\mu, \sigma^2)$ | `norm(loc=volumes.mean(), scale=std_calc(...))`; overlap grid clipped at 0 after probabilities are computed |
| Mean update | $\mu_t = \frac{1}{t+1}\sum_{\tau=0}^{t} \delta_\tau$ | $\mu_t = \frac{1}{t+1}\sum_{\tau=0}^{t} o_\tau$ | `volumes.mean()` inside each solver call |
| Variance / standard deviation update | MAP estimate with gamma prior over $\sigma$ | Same idea for overlap standard deviation | `std_calc(measured_data, alpha, beta)` with defaults from `constants.py` |
| State variable in MDP equations | Base model: $s_t = (\delta_t, B_N)$, i.e. 2 state coordinates if time is handled separately; appendix: $(\delta_t, B_N, B_T)$, i.e. 3 state coordinates if time is handled separately | $s_t = (o_t, D_t)$, i.e. 2 state coordinates if time is handled separately | Online decision variables are effectively $(o_t, D_t)$ plus the clinical fraction index outside the tuple; stored DP array is 3D: time-to-go $\times$ accumulated dose $\times$ overlap |
| Estimated distribution parameters inside state? | No. $\mu_t$ and $\sigma_t$ are updated from observations, but not added as explicit Bellman state coordinates in the main formulation. | No. $\mu_t$ and $\sigma_t$ are re-estimated during treatment, but the paper explicitly keeps them outside the MDP state used in the recursion. | No. `volumes.mean()` and `std_calc(...)` are recomputed once per solver call and then treated as fixed model inputs for that solve. |
| Action bounds | $d_t \in [d_{\min}, d_{\max}]$ | $d_t \in [d_{\min}, d_{\max}]$ | `DEFAULT_MIN_DOSE = 6`, `DEFAULT_MAX_DOSE = 10`, clipped further by `max_action(...)` |

| Concept | 2023 paper | 2025 paper | Current codebase |
|---|---|---|---|
| Policy object | $\pi_t(\delta, B_N)$ or $\pi_t(\delta, B_N, B_T)$ | $\pi_t(o, D)$ | `policies` stores full future policy grid; `policies_overlap` stores current-fraction policy over overlap grid |
| Value object | $v_t(\delta, B_N)$ or appendix extension | $v_t(o, D)$ | `values[state, dose_index, volume_index]` |
| Time indexing | Clinical fraction index $t$ | Clinical fraction index $t$ | Backward-recursion index `state` counts time-to-go, not clinical time directly |
| Prescription target | OAR BED limit $B_N^{\max}$ drives feasible set | Exact total prescribed physical dose $D_{\text{pres}}$ | `goal = number_of_fractions * mean_dose` |

# 4 Equation-Level Translation

## 4.1 2023 Paper

The 2023 paper formulates adaptive fractionation in BED space. The geometry term is the sparing factor $\delta_t$, the state tracks cumulative OAR BED, and the immediate stage term is a *reward* in tumor BED:

$$s_t^{2023} = (\delta_t, B_N), \tag{1}$$

$$r_t^{2023}(d_t) = d_t + \frac{d_t^2}{(\alpha/\beta)_T}. \tag{2}$$

Written with time explicit, the base 2023 Bellman state is therefore

$$\text{state}_{2023,\text{base}} = (t, \delta_t, B_N), \tag{3}$$

which is a finite-horizon process with two non-time state coordinates. In the appendix, cumulative tumor BED is added: *In simple words.* In the main 2023 model, the algorithm needs to know three things at decision time: which fraction we are in, what today's sparing factor is, and how much OAR BED has already been accumulated.

$$\text{state}_{2023,\text{appendix}} = (t, \delta_t, B_N, B_T), \tag{4}$$

which raises the non-time state dimensionality from 2 to 3. *In simple words.* The appendix adds one more memory variable: how much tumor BED has already been delivered. That makes the policy depend on both OAR history and tumor history.

The Bellman recursion is therefore a maximization over expected future reward subject to a terminal OAR BED constraint:

$$v_t(\delta, B_N) = \max_d \left[ d + \frac{d^2}{(\alpha/\beta)_T} + \sum_{\delta'} P(\delta') v_{t+1}\left(\delta', B_N + \delta d + \frac{\delta^2 d^2}{(\alpha/\beta)_N}\right) \right]. \tag{5}$$

*In simple words.* At each fraction, the 2023 policy chooses the dose that gives the best tumor-BED reward now plus the best expected reward later, while accounting for the OAR BED that this dose adds.

The appendix then shows a state extension in which cumulative tumor BED is added explicitly. That appendix is the closest paper-level precedent for increasing state dimensionality in this repository.

### 4.2 2025 Paper

The 2025 paper changes the geometry descriptor from sparing factor to overlap volume and changes the optimization target from tumor-BED reward to overlap-based cost minimization:

$$s_t^{2025} = (o_t, D_t), \tag{6}$$

$$c_t^{2025}(o_t, d_t) = o_t(d_t - d_{\min}). \tag{7}$$

Written with time explicit, the 2025 Bellman state is

$$\text{state}_{2025} = (t, o_t, D_t), \tag{8}$$

which is again a finite-horizon process with two non-time state coordinates. The important point is that the paper does *not* include $(\mu_t, \sigma_t)$ inside this state tuple. They are updated between fractions and then used to define the transition model for the next solve, but the recursion itself is written only over overlap and accumulated dose. *In simple words.* In the 2025 model, the policy needs to know what fraction we are in, how much overlap we see today, and how much dose has already been delivered. It does not treat the estimated mean and standard deviation as state variables.

The 2025 paper also contains a second mathematical object that is easy to confuse with the treatment loss: its equation (6) is the MAP estimator for $\sigma_t$, not the stage cost and not the Bellman objective. In paper notation,

$$\sigma_t = \arg\max_{\sigma} \; p(\sigma \mid o_0, \ldots, o_t). \tag{9}$$

So equation (6) does *not* minimize treatment loss at all. It maximizes a posterior density over possible values of the future-overlap standard deviation. Its role is purely to choose the parameter that shapes the next-step overlap distribution $P(o')$ used inside dynamic programming. *In simple words.* Read this as: "given the overlap values we have seen so far, choose the standard deviation that looks most plausible." It is an uncertainty-fitting step, not a dose-selection step.

**How the paper updates $\mu_t$ and $\sigma_t$ from fraction to fraction.** The paper's online update loop is easiest to read as a short sequence:

1. Start fraction $t$ with the overlap history observed so far, $o_0, \ldots, o_t$.

2. Recompute the running mean

$$\mu_t = \frac{1}{t+1} \sum_{\tau=0}^{t} o_\tau.$$

   *In simple words.* This is just the average of all overlap values seen so far. Early in treatment, one new overlap can move this average a lot. Later in treatment, the same new overlap moves it less because it is being averaged together with more past observations.

3. Recompute the patient-specific empirical spread $\sigma_t^{\text{pat}}$ from that same history. *In simple words.* The paper also recomputes how spread out the patient's observed overlaps have been. As more fractions are observed, this spread estimate usually becomes more stable as well.

4. Combine $\sigma_t^{\text{pat}}$ with the gamma prior to obtain the MAP estimate $\sigma_t$ via equation (6).

5. Use the updated pair $(\mu_t, \sigma_t)$ to define the distribution of the next overlap, $o_{t+1} \sim \mathcal{N}(\mu_t, \sigma_t^2)$. *In simple words.* After updating the mean and standard deviation from the overlaps seen so far, the paper uses those two numbers as its current model for the next fraction's overlap.

6. Solve the Bellman recursion for the current fraction under that fixed transition model.

7. After the next fraction is observed, append the new overlap to the history and repeat the update.

*In simple words.* The paper is doing: "observe one more overlap, update the mean, update the uncertainty, then solve today's dose decision using those updated beliefs about the future."
*Code pointers.* In the repository, the same update appears in the core solver module `core_adaptfx.py`. That module lives in package `adaptive_fractionation_overlap`. Specifically, `std = std_calc(volumes, alpha, beta)` appears at line 161, followed by `distribution = norm(loc = volumes.mean(), scale = std)` at line 162. The underlying MAP-style standard-deviation calculation is implemented in `helper_functions.py` from the same package, lines 48–79. A second copy of the same update appears in `precompute_plan(...)` at lines 313–314 of `core_adaptfx.py`. The code-level analogue of "append the newly observed overlap and repeat" can be seen in line 321 of `core_adaptfx.py`, where `np.append(volumes, volume)` is passed into `adaptive_fractionation_core(...)`.

That separation is important:

- paper equation (1) defines the one-step loss, $c_t = o_t(d_t - d_{\min})$,
- paper equation (6) estimates $\sigma_t$,
- paper equation (10) minimizes expected total future loss using the current estimates of $\mu_t$ and $\sigma_t$.

The quantity that the policy actually minimizes is therefore not "equation (6)", but the expected cumulative overlap-weighted underdosage cost:

$$\mathbb{E}\left[\sum_{\tau=t}^{F} o_\tau(d_\tau - d_{\min}) + c_{F+1}\right]. \tag{10}$$

Equation (6) matters only indirectly: a larger estimated $\sigma_t$ spreads probability mass over a wider range of future overlaps, which can make the current action more conservative or more opportunistic depending on the remaining dose budget. *In simple words.* The real goal is: choose today's dose so that the total overlap penalty from today to the end of treatment is as small as possible, while still finishing the prescription.

Its Bellman recursion is therefore a minimization:

$$v_t(o, D) = \min_{d \in [d_{\min}, d_{\max}]} \left[o(d - d_{\min}) + \sum_{o'} P(o') v_{t+1}(o', D + d)\right]. \tag{11}$$

*In simple words.* At each fraction, the 2025 policy looks for the daily dose that minimizes two things added together: the penalty you pay today because of today's overlap, and the expected penalty you will pay in the remaining fractions.

The terminal condition enforces exact delivery of the prescribed total dose:

$$c_{F+1} = \begin{cases} 0 & \text{if the final cumulative dose reaches } D_{\text{pres}}, \\ \infty & \text{otherwise.} \end{cases} \tag{12}$$

*In simple words.* If the full prescribed dose has been delivered by the end, there is no extra penalty. If not, the plan is treated as infeasible.

### 4.3 Current Code

The current code is closest in spirit to the 2025 paper, but not identical to it. The solver in `adaptive_fractionation_core(...)` estimates

$$\hat{\mu}_t = \text{mean}(\texttt{volumes}), \tag{13}$$

$$\hat{\sigma}_t = \texttt{std_calc(volumes, alpha, beta)}, \tag{14}$$

builds a Gaussian model for future overlap, discretizes overlap and accumulated dose, and solves a backward dynamic program. *In simple words.* The code follows the same outer loop as the paper: after each new observed fraction, recompute the mean and standard deviation from the full observed history, then solve the current fraction using those fixed values.

Written with time explicit, the implemented online decision state is best understood as

$$\text{state}_{\text{impl,online}} = (t, o_t, D_t), \tag{15}$$

while the stored DP tensor is indexed as

$$\text{state}_{\text{impl,array}} = (\text{time-to-go}, D, o). \tag{16}$$

So the implementation is solving a finite-horizon process with two non-time state coordinates, but it stores time as an explicit array axis rather than burying it in the notation. *In simple words.* Conceptually, the code is solving the same kind of problem as the 2025 paper: "what do I do at this fraction, for this overlap, given the dose already delivered?" Internally, it stores that as a 3D table with time as a separate axis.

The implemented one-step penalty is

$$\hat{c}_t(o, d) = o(d - d_{\min}) + \frac{1}{2}\,|\texttt{INTERCEPT} + \texttt{SLOPE} \cdot o|\,(d - d_{\min})^2, \tag{17}$$

as encoded in `penalty_calc_single(...)` and `penalty_calc_matrix(...)`. This is stricter than the 2025 paper's base linear cost. *In simple words.* The code penalizes giving extra dose on overlap days twice: once with the paper's basic linear penalty, and again with an extra quadratic penalty that grows faster when the chosen dose moves further above $d_{\min}$.

Because the code stores values as utilities to be maximized, the implemented recursion is best read as

$$V_t^{\text{impl}}(o, D) = \max_d \left[ -\hat{c}_t(o, d) + \mathbb{E}_{o'} V_{t+1}^{\text{impl}}(o', D + d) \right], \tag{18}$$

with infeasible end states given a very large negative value rather than a symbolic $\infty$. *In simple words.* The code is still doing cost minimization in spirit. It just flips the sign so the solver can maximize "negative cost now plus best expected value later."

# 5 Semantic Mismatches That Matter

## 5.1 2023 Paper vs. 2025 Paper

The 2023 and 2025 papers are not minor notational variants of the same MDP.

- The geometry descriptor changes from sparing factor $\delta_t$ to overlap volume $o_t$.
- The accumulated state changes from OAR BED $B_N$ to cumulative delivered physical dose $D_t$.
- The objective changes from maximizing tumor BED to minimizing overlap-driven underdosage.
- The terminal feasibility condition changes from an OAR-BED ceiling to an exact tumor-dose target.

This means the 2025 paper is not just "the 2023 paper with renamed symbols"; it is a different control problem motivated by a different clinical bottleneck.

## 5.2 2025 Paper vs. Code

The code is again not a literal transcription of the 2025 paper.

- The paper's stage cost is linear in overlap and excess dose. The code adds the quadratic correction in equation (17).
- The paper writes the MDP state as $(o_t, D_t)$. The code's solved object is effectively indexed by time-to-go, accumulated-dose grid, and overlap grid.
- The paper discusses $\mu_t$ and $\sigma_t$ as updated during treatment. The code updates them once per solver call and then treats them as fixed model parameters for that solve.
- The paper writes a clean terminal cost. The code uses the sentinel value `-1000000000000` to represent infeasible terminal states.
- The code computes probabilities on the pre-clipped Gaussian grid and only then clips negative overlap states to zero, so the implemented distribution handling is numerically convenient rather than mathematically exact.

## 5.3 Paper State vs. Implemented State

The cleanest way to compare dimensionality is to separate "time" from "state coordinates that the policy conditions on":

- 2023 main text: 2 non-time coordinates, $(\delta_t, B_N)$.
- 2023 appendix: 3 non-time coordinates, $(\delta_t, B_N, B_T)$.
- 2025 paper: 2 non-time coordinates, $(o_t, D_t)$.
- current code: 2 non-time coordinates in the decision rule, $(o_t, D_t)$, plus one explicit DP axis for time-to-go.

If time is written explicitly, the implemented finite-horizon decision process is most naturally described as

$$\text{state}_{\text{impl}} = (t, D_t, o_t), \tag{19}$$

with distribution parameters $(\mu_t, \sigma_t)$ estimated from the observed history but not represented as explicit state coordinates inside the DP table. *In simple words.* If you say the state out loud, it is: "which fraction is this, how much dose have we already used, and how much overlap do we see right now?"

At the array level, the current implementation uses

$$\text{DP tensor shape} = \text{time-to-go} \times \text{accumulated dose} \times \text{overlap}. \tag{20}$$

*In simple words.* The solver stores one value table entry for every combination of remaining time, accumulated dose bucket, and overlap bucket.

That distinction matters because a future extension can change either

- the *online state used by the policy*, or
- the *state dimensions explicitly enumerated in the DP tables*,

and those are related but not identical design choices.

# 6 Extension Implications

If the intended extension is to make overlap mean and standard deviation explicit state variables, then the natural finite-horizon mathematical state becomes

$$s_t^{\text{extended}} = (o_t, D_t, \mu_t, \sigma_t) \tag{21}$$

*In simple words.* This extension would mean the policy no longer depends only on today's overlap and the delivered dose so far. It would also depend explicitly on what the model currently believes about the average future overlap and how uncertain that belief still is.

The main conceptual gain is not merely higher dimensionality. The main gain is that the decision process can represent *learning during treatment.* In the current implementation, the solver estimates

$$\hat{\mu}_t = \text{mean}(\texttt{volumes}), \tag{22}$$

$$\hat{\sigma}_t = \texttt{std_calc(volumes, alpha, beta)}, \tag{23}$$

once at the start of the solve and then treats the future overlap distribution as fixed for all remaining fractions in that solve. In code, this happens at lines 161–162 of `core_adaptfx.py` in package `adaptive_fractionation_overlap`. The current Bellman recursion therefore distinguishes early and late fractions mainly by remaining time and remaining dose budget, not by how much confidence has been gained about the patient's personal overlap distribution.

*In simple words.* Right now the code says: "based on what I know today, I will assume the same overlap distribution for the rest of this solve." It does not explicitly reason about the fact that after the next fraction we will have learned more.

If $(\mu_t, \sigma_t)$ become part of the state and are updated by observed overlaps, then the next state carries not only a new overlap and a new cumulative dose, but also a new belief about future overlaps. This changes the meaning of the value function. The recursion is no longer evaluating only

> "what is the best dose if today's overlap is $o_t$ and the delivered dose so far is $D_t$?"

It is also evaluating

> "what is the best dose given what I currently believe about this patient's future overlap distribution, knowing that belief itself will be updated after the next observation?"

*In simple words.* The state would start to encode not just the patient's geometry, but also the algorithm's current level of knowledge about that geometry.

That matters clinically and algorithmically because later fractions should be more informed than earlier fractions. After more observations, the patient-specific overlap model should usually be less uncertain. A richer state can therefore support policies with behavior such as:

- being more cautious in early fractions, because the estimated overlap distribution is still unstable,
- preserving dose flexibility while the model is still learning whether this patient tends to have favorable or unfavorable geometry,
- acting more decisively in later fractions, because the belief about the overlap distribution has been refined by additional measurements.

*In simple words.* With this extension, the algorithm could explicitly reason: "I am still uncertain, so I may want to wait," versus "I now know this patient's pattern better, so I can commit more confidently."

This is also the point at which the extension becomes more than a reparameterization. The transition model would become a *belief update* model: observing the next overlap would update the physical-treatment part of the state $(o_t, D_t)$ and the statistical-belief part of the state $(\mu_t, \sigma_t)$.

Relative to the current implementation, that is not a cosmetic patch. It changes:

- what information the policy conditions on,
- the transition model, because $(\mu_t, \sigma_t)$ would now update as part of the controlled process,
- the interpretation of the value function, which would now include the value of future information,
- the memory and runtime complexity,
- the policy object, which would no longer be a function of overlap and accumulated dose alone.

The coherent interpretation used in this note is that the triple $(t, \mu_t, \sigma_t)$ already carries a practical notion of certainty. Time $t$ tells us how many overlap observations have been incorporated. The pair $(\mu_t, \sigma_t)$ summarizes the current fitted overlap model. The update rule then makes those summaries progressively less sensitive to one new observation as treatment proceeds.

For the mean, this can be written recursively as

$$\mu_{t+1} = \mu_t + \frac{o_{t+1} - \mu_t}{t+2}.$$

*In simple words.* Start from today's mean estimate. Then move it part of the way toward the newly observed overlap. That step size is $1/(t+2)$, so early fractions can change the estimate a lot, while later fractions change it only a little.

This means that if the state contains $(t, \mu_t, \sigma_t)$ and the transition law updates those quantities in the same way as the estimation procedure, then the model is already carrying an operational notion of confidence: later belief states are harder to move because they are based on more accumulated evidence.

*In simple words.* The certainty is not stored as a separate variable called "confidence." Instead, it shows up in how stable the belief state has become. Early on, one new observation can change the state a lot. Later on, the same observation changes it much less.

Under this interpretation, a separate confidence variable is not required for a first implementation. The state itself, together with its update law, already represents learning over time. This representation is estimator-based rather than fully Bayesian: it tracks the current fitted parameters and how stable they have become, not a full probability distribution over all plausible parameter values.

*In simple words.* A 5D state can already do the job if we define it carefully. It captures the idea that our belief becomes more stable over time. Extra posterior variables would only be needed later if we decide we want a stricter Bayesian description of uncertainty, rather than this practical estimator-based one.

The 2023 appendix is useful here because it already demonstrates the logic of promoting an accumulated quantity into the state. The same pattern can be reused conceptually when deciding whether $(\mu_t, \sigma_t)$ should remain estimated parameters or become true state variables.

# 7 Code Mapping Appendix

| **Code symbol / function** | **Closest paper concept** | **Role in implementation** |
|---|---|---|
| `fraction` | Time index $t$ | Clinical fraction currently being solved |
| `volumes` | $\{o_\tau\}$ | Observed overlap history up to the current call |
| `actual_volume` | $o_t$ | Most recently observed overlap, i.e. the current geometry |
| `accumulated_dose` | $D_t$ | Cumulative delivered physical dose before current fraction |
| `goal` | $D_{\text{pres}}$ | Total prescribed physical dose, defined as `number_of_fractions * mean_dose` |
| `std_calc(...)` | $\sigma_t$ update | Computes a MAP-like standard-deviation estimate using a gamma prior |
| `distribution` | $P(o)$ or $P(\delta)$ | Gaussian model for future overlap values in the current solve |
| `volume_space` | Discretized overlap state | 200-point overlap grid from the estimated Gaussian |
| `probabilities` | $P(o')$ | Probability masses attached to the overlap grid |
| `dose_space` | Discretized $D_t$ | Grid of accumulated physical-dose states |
| `delivered_doses` | Action space | Candidate per-fraction doses between `min_dose` and `max_dose` |

| Code symbol / function | Closest paper concept | Role in implementation |
|---|---|---|
| `max_action(...)` | Feasibility restriction | Removes actions that would overshoot the total goal |
| `values` | $v_t(\cdot)$ or $V_t(\cdot)$ | DP value tensor over future states |
| `policies` | $\pi_t(\cdot)$ | DP policy tensor over future states |
| `policies_overlap` | Slice of $\pi_t$ at current accumulated dose | Current-fraction dose recommendation as a function of overlap |
| `physical_dose` | Chosen action $d_t$ | Actual dose selected for the observed current overlap |
| `penalty_calc_single(...)` | Immediate cost | Scores one dose-overlap pair |
| `penalty_calc_matrix(...)` | Immediate cost over grid | Scores all dose-overlap combinations used in DP |
| `adaptfx_full(...)` | Policy rollout | Applies the single-fraction solver sequentially over a full treatment course |
| `precompute_plan(...)` | Policy sweep over hypothetical $o_t$ | Builds a lookup table from overlap value to recommended dose |
| `policy_calc(...)` | Fixed-distribution variant | Solves the same DP using externally supplied mean and standard deviation |

# 8 Practical Reading Rule

When reading or modifying this repository, the safest translation is:

1. Start from the 2025 paper, because it is the nearest conceptual ancestor of the current solver.
2. Treat the 2023 appendix as the template for "what it means to add a state dimension".
3. Treat the current Python code as a *paper-inspired implementation*, not as a verbatim encoding of either paper.

That framing keeps the next design step honest: if we extend the MDP, we should state clearly whether we are following a paper exactly, borrowing a paper's structural idea, or deliberately introducing a new model.